Posts and Telecommunications Institute of Technology

ASSIGNMENT 1: Bank marketing

INTELLIGENT SYSTEM DEVELOPMENT

Lecturer : TRẦN ĐÌNH QUẾ

Student : MAI ANH TUẤN ANH

Student ID : B23DCCE005

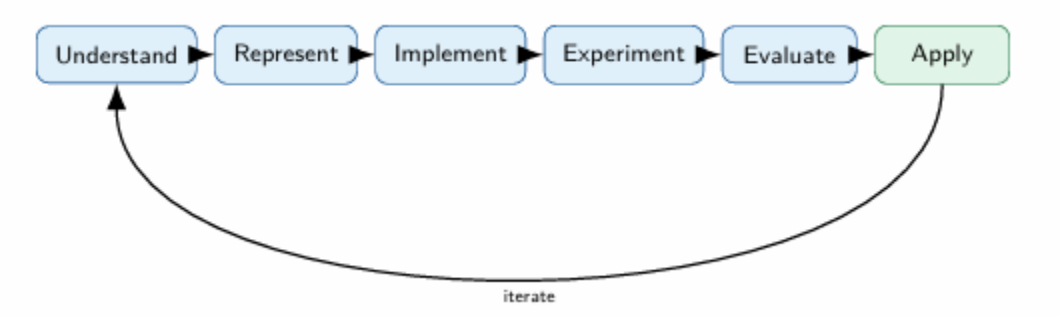
Class : E23CNPM02

Hà Nội, August 2026

1. Introduction

- The purpose of this assignment is not simply to train a machine learning model, but to develop and explain a small intelligent system from a real-world problem.

This assignment is a progression from understanding intelligence to building systems that can represent information, learn from data, make decisions, act, and adapt. The development process is summarized as:



- Besides, an intelligent system is more than a trained machine learning model. A complete system connects data and computational methods to an environment through representation, interfaces, interaction, evaluation, deployment, and feedback.

For this assignment , the course starts with a relatively simple setting:

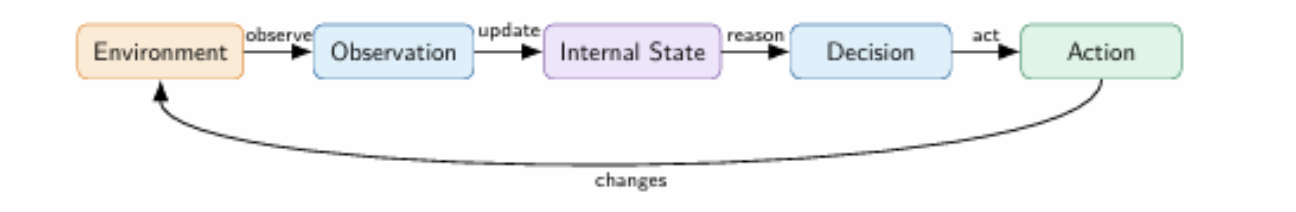
Structured data → Feature vector → Traditional ML → Application

2. Intelligent System Definition

- An intelligent system can be understood as a computational system that interacts with an environment, processes information through an internal representation, learns or reasons from that information, makes decisions, and produces actions or outputs. In this view, intelligence is not associated with a single algorithm. It emerges from the coordination of several capabilities, including perception, representation, learning, reasoning, decision making, and action.

- A useful abstraction of an intelligent system is:

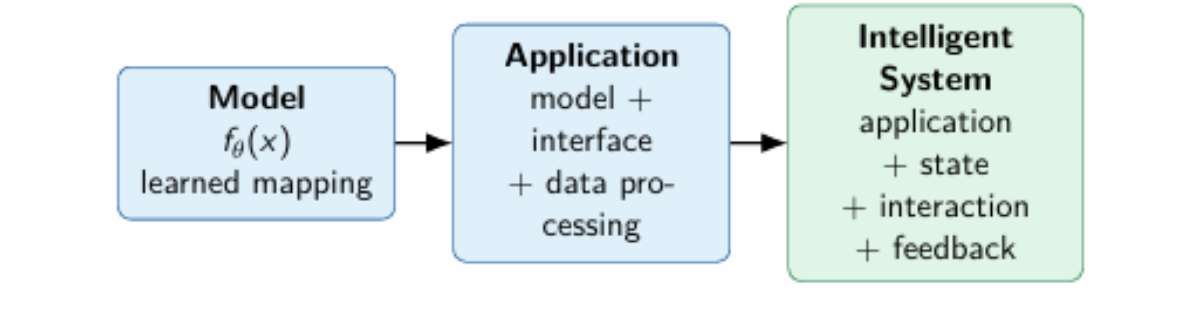
Intel Syst =
(Environment,State,Representation,Knowledge,Learning,Decision,Action)



- The environment contains the external information with which the system interacts. The system does not have direct access to the complete state of the environment. Instead, it receives observations and constructs an internal representation of the information that is currently available. This distinction between external state and internal state is important because the information available to the learning process depends on how the environment is represented.

Representation therefore plays a central role in intelligent system development. A learning algorithm does not operate directly on a real-world object, such as a customer, image, or document. It operates on a computational representation of selected information. Different representations preserve different information and consequently support different learning mechanisms.

An intelligent system should also be distinguished from a machine learning model or an application. A model is a learned function that maps an input representation to an output. An application combines the model with data processing and an interface so that the model can be used in practice. An intelligent system represents a broader structure in which the application operates within an environment and may incorporate state, interaction, knowledge, and feedback.



Therefore, the development of an intelligent system is not limited to model training. It involves a complete process of understanding the problem, determining

what information is available, selecting an appropriate representation, implementing a learning mechanism, conducting experiments, evaluating the results, deploying the resulting system, and iterating based on evidence.

3. Problem Definition

- The problem is to predict whether a client will subscribe to a term deposit after being contacted during a bank marketing campaign.

- The formal problem statement is:

Given the selected feature vector of a previously unseen client, predict the target class of that client.

- This is a supervised binary classification problem.

For each observation, the system has an input representation x and a target y. The learning problem can be expressed as:

$D = \{(xi, yi)\}$

where xi is the input feature representation and yi is the corresponding target.

The model learns a function:

$\hat{y} = f\theta x$

where θ represents the learned parameters of the model and $\hat{y}$ is the predicted class.

Given:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N},$$

we seek:

$$\hat{y} = f_\theta(x).$$

For classification:

$$\hat{y} \in \{0, 1\}.$$

The learning process searches for:

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell\left(f_\theta(x_i), y_i\right).$$

For this project: y =

0 → no subscription

1 → subscription

- This formulation follows the assignment requirement that students explicitly define the input, target, learning task, prediction, training, testing, and generalization.

The data is divided into a training set and a test set. The training set is used for learning, while the test set is kept unseen during model development and is used to estimate performance on new observations.

4. Dataset

- The project uses the Bank Marketing dataset obtained from Kaggle. The dataset represents direct marketing campaigns conducted by a Portuguese banking institution.

Here is its source: [Bank Marketing Data Set | Kaggle]

The file used in the project is:

bank-additional-full.csv

- The dataset contains 41,188 observations and a combination of numerical and categorical variables.

```
File: bank-additional-full.csv
Shape: (41188, 21)
```

The features describe several aspects of the customer and the marketing campaign. They include demographic information such as age, job, marital status, and education, financial information such as housing and personal loans, information about campaign contacts, previous campaign outcomes, and several economic indicators.

Feature	Type	Meaning
age	Numerical	Age of the client
job	Categorical	Type of job
marital	Categorical	Marital status
education	Categorical	Education level
default	Categorical	Whether the client has credit in default
housing	Categorical	Whether the client has a housing loan
loan	Categorical	Whether the client has a personal loan
contact	Categorical	Communication type
month	Categorical	Month of the last contact
day_of_week	Categorical	Day of the week of the last contact
campaign	Numerical	Number of contacts performed during the current campaign
pdays	Numerical	Number of days since the client was last contacted from a previous campaign
previous	Numerical	Number of contacts performed before the current campaign
poutcome	Categorical	Outcome of the previous marketing campaign
emp.var.rate	Numerical	Employment variation rate
cons.price.idx	Numerical	Consumer price index
cons.conf.idx	Numerical	Consumer confidence index
euribor3m	Numerical	3-month Euribor rate
nr.employed	Numerical	Number of employees

The dataset is appropriate for this assignment because it represents a real-world problem, contains identifiable observations and a clearly defined target, provides

both numerical and categorical features, and is suitable for traditional machine learning.

- One important modeling decision was to exclude duration from the main representation. Duration describes the length of the last contact. For a system intended to support prediction before the interaction has finished, using this variable would introduce information that is not available at the intended prediction time. Therefore, it was excluded from the main feature representation.

5. Data Representation

- The machine learning algorithm does not receive the real-world client directly. So that, the client is converted into a computational representation. For structured data, this representation is a feature vector:

x = [x1, x2, ..., xd]

For this project, the raw bank marketing record contains both numerical and categorical information.

```
Dataset loaded successfully.
File: bank-additional-full.csv
Shape: (41188, 21)

Numerical features:
['age', 'campaign', 'pdays', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed']

Categorical features:
['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']

Target:
y
0    36548
1     4640
Name: count, dtype: int64
```

- The preprocessing procedure is:

Raw information → Selected features → Numerical and categorical preprocessing → Numerical feature vector → ML model

=> Numerical features are standardized. Categorical features are converted using one-hot encoding.

This means that a categorical variable such as job is not passed to the model as a text value. Instead, its categories are transformed into numerical indicator features.

Therefore, the representation used by the model is different from the original raw record.

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", encoder, categorical_features)
    ],
    remainder="drop"
)
```

	Feature	Type	Model Representation
0	age	Numerical	Standardized numerical value
1	job	Categorical	One-hot encoded categorical value
2	marital	Categorical	One-hot encoded categorical value
3	education	Categorical	One-hot encoded categorical value
4	default	Categorical	One-hot encoded categorical value
5	housing	Categorical	One-hot encoded categorical value
6	loan	Categorical	One-hot encoded categorical value
7	contact	Categorical	One-hot encoded categorical value
8	month	Categorical	One-hot encoded categorical value
9	day_of_week	Categorical	One-hot encoded categorical value
10	campaign	Numerical	Standardized numerical value
11	pdays	Numerical	Standardized numerical value
12	previous	Numerical	Standardized numerical value
13	poutcome	Categorical	One-hot encoded categorical value
14	emp.var.rate	Numerical	Standardized numerical value
15	cons.price.idx	Numerical	Standardized numerical value
16	cons.conf.idx	Numerical	Standardized numerical value
17	euribor3m	Numerical	Standardized numerical value
18	nr.employed	Numerical	Standardized numerical value

The choice of representation is important because it determines what information is available to the learner.

This idea is also examined experimentally later in the project by comparing scaled and unscaled numerical representations for KNN.

6. Traditional ML Methods

- Four traditional machine learning models were trained:

+ Logistic Regression

+ K-Nearest Neighbors

+ Decision Tree

+ Random Forest

The purpose of using several models is not simply to find the model with the highest score. The four models use the same basic feature representation but apply different learning mechanisms and assumptions. This is an important characteristic of traditional machine learning: different algorithms can receive essentially the same representation while learning different relationships from it.

- Logistic Regression

Logistic Regression learns a weighted relationship between the feature vector and the probability of the positive class.

For binary classification, the model computes:

$$z = w^T x + b, \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

where σ is the sigmoid function.

Each feature contributes through a learned weight. A decision threshold is then used to convert the probability-like score into a class prediction.

In this project, the model receives the preprocessed feature vector containing standardized numerical features and one-hot encoded categorical features.

The learned parameters are the feature weights and the intercept.

```
from sklearn.linear_model import LogisticRegression

model_1 = Pipeline([
    ("preprocess", preprocessor),
    (
        "model",
        LogisticRegression(
            max_iter=1000,
            random_state=RANDOM_STATE
        )
    )
])

model_1.fit(X_train, y_train)

y_pred_1 = model_1.predict(X_test)

print("Model 1: Logistic Regression")
print("Model trained successfully.")
print("Number of test predictions:", len(y_pred_1))
```

```
Model 1: Logistic Regression
Model trained successfully.
Number of test predictions: 8238
```

=> Logistic Regression is relatively simple and interpretable, but its main limitation is that it represents the decision relationship through a relatively simple weighted structure.

- K-Nearest Neighbors

KNN follows a different idea. Instead of learning an explicit weighted function, it classifies a new observation using nearby observations from the training set.

Using Euclidean distance, the distance between two feature vectors is based on the difference between their feature values. The closest k observations are used to determine the predicted class.

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}.$$

The important assumption is that observations that are close to each other in the feature space tend to have similar target classes.

Because KNN depends on distance, feature scale is important. This is why numerical features are standardized in the main representation.

The main hyperparameter investigated in this project is k.

```
from sklearn.neighbors import KNeighborsClassifier

model_2 = Pipeline([
    ("preprocess", preprocessor),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=19
        )
    )
])

model_2.fit(X_train, y_train)

y_pred_2 = model_2.predict(X_test)

print("Model 2: K-Nearest Neighbors")
print("Initial k =", 19)
print("Model trained successfully.")
print("Number of test predictions:", len(y_pred_2))
```

```
Model 2: K-Nearest Neighbors
Initial k = 19
Model trained successfully.
Number of test predictions: 8238
```

=> KNN is simple and can model nonlinear local relationships, but it can be sensitive to the choice of k and to the representation of the data.

- Decision Tree

Decision Tree recursively divides the feature space using selected splitting criteria.

The model learns a tree structure consisting of decision rules, internal split nodes, and leaf nodes. Each split attempts to produce groups that are more homogeneous with respect to the target class.

```python
from sklearn.tree import DecisionTreeClassifier

model_3 = Pipeline([
    ("preprocess", preprocessor),
    (
        "model",
        DecisionTreeClassifier(
            max_depth=8,
            min_samples_leaf=10,
            random_state=RANDOM_STATE
        )
    )
])

model_3.fit(X_train, y_train)

y_pred_3 = model_3.predict(X_test)

print("Model 3: Decision Tree")
print("max_depth =", 8)
print("min_samples_leaf =", 10)
print("Model trained successfully.")
print("Number of test predictions:", len(y_pred_3))
```

```
Model 3: Decision Tree
max_depth = 8
min_samples_leaf = 10
Model trained successfully.
Number of test predictions: 8238
```

=> Decision Trees can model nonlinear relationships and interactions between features. They are also relatively easy to interpret.

However, a tree that becomes too complex may fit the training data too closely and generalize poorly to unseen observations.

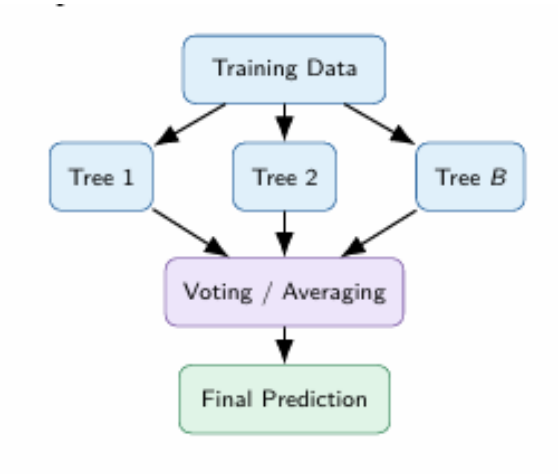
- Random Forest

Random Forest extends the Decision Tree idea by combining many trees.

Each tree is trained using a bootstrap sample, and each split considers a random subset of features. The final prediction is obtained by combining the predictions of the trees.

The basic idea can be summarized as:

Many trees → Combined prediction




```python
from sklearn.ensemble import RandomForestClassifier

model_4 = Pipeline([
    ("preprocess", preprocessor),
    (
        "model",
        RandomForestClassifier(
            n_estimators=200,
            max_depth=12,
            min_samples_leaf=5,
            random_state=RANDOM_STATE,
            n_jobs=-1
        )
    )
])

model_4.fit(X_train, y_train)

y_pred_4 = model_4.predict(X_test)

print("Model 4: Random Forest")
print("n_estimators =", 200)
print("max_depth =", 12)
print("min_samples_leaf =", 5)
print("Model trained successfully.")
print("Number of test predictions:", len(y_pred_4))
```

```
Model 4: Random Forest
n_estimators = 200
max_depth = 12
min_samples_leaf = 5
Model trained successfully.
Number of test predictions: 8238
```

=> The main strength of Random Forest is that multiple diverse trees can provide a more robust prediction than a single decision tree.

However, the main disadvantages are greater computational cost and lower interpretability compared with a single tree.

Overall, these four models provide four different learning mechanisms over the same general feature-vector representation.

7. Experimental Design

The goal is not simply to run several models and report a number. A stronger experiment asks why the result occurred and investigates meaningful changes in the model, hyperparameters, or representation.

Three controlled experiments were conducted.

- Experiment 1: Model Comparison

The first experiment compares the four traditional machine learning models using the same training data, preprocessing procedure, and evaluation protocol.

The experimental question is:

+ Under the same preprocessing and the same 5-fold cross-validation protocol, which of the four traditional ML models provides the best overall classification performance?

```
for name, model in models.items():

    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring=[
            "accuracy",
            "precision",
            "recall",
            "f1"
        ],
        n_jobs=-1
    )

    experiment_1_results.append({
        "Model": name,
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1": scores["test_f1"].mean()
    })
```

This experiment provides the main evidence used for model selection.

	Model	Accuracy	Precision	Recall	F1
0	Decision Tree	0.8975	0.6072	0.2581	0.3620
1	KNN	0.8988	0.6283	0.2511	0.3584
2	Random Forest	0.8999	0.6643	0.2263	0.3372
3	Logistic Regression	0.8995	0.6564	0.2268	0.3370

The model with the highest F1-score is preferred for the main application because the positive class is relatively rare and the application requires a balance between finding potential subscribers and avoiding excessive false positives.

The results should therefore be interpreted using multiple metrics rather than Accuracy alone.

Best model according to cross-validated F1: Decision Tree

- Experiment 2: Hyperparameter Investigation

The second experiment investigates the KNN hyperparameter k.

The experimental question is:

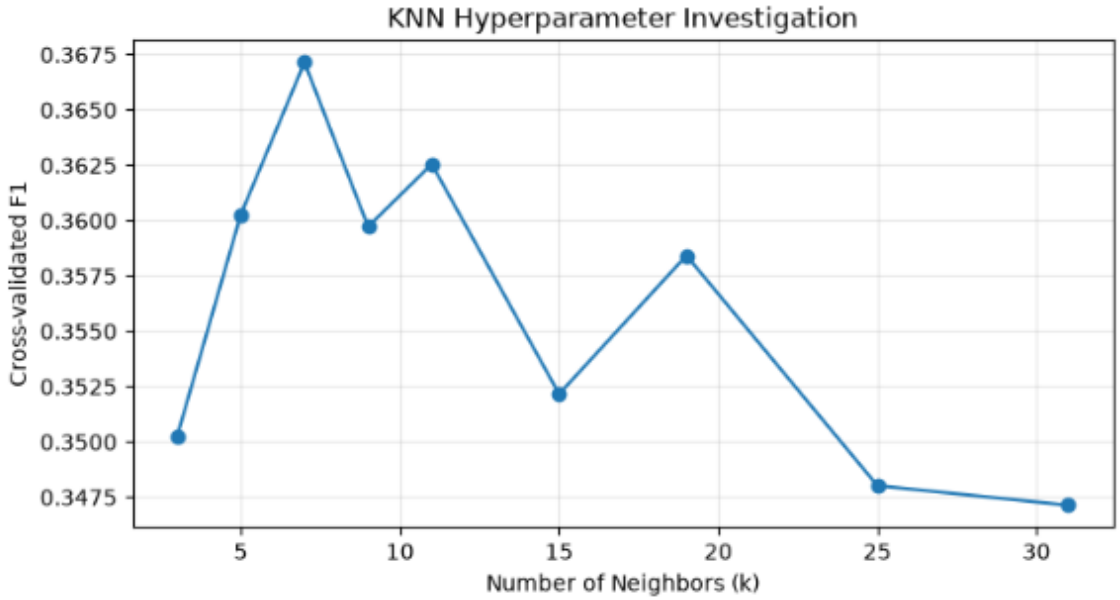
+ How does the KNN hyperparameter k affect classification performance?

```
for k in k_values:

    knn_model = Pipeline([
        ("preprocess", preprocessor),
        (
            "model",
            KNeighborsClassifier(
                n_neighbors=k
            )
        )
    ])

    scores = cross_validate(
        knn_model,
        X_train,
        y_train,
        cv=cv,
        scoring=[
            "accuracy",
            "precision",
            "recall",
            "f1"
        ],
        n_jobs=-1
    )

    experiment_2_results.append({
        "k": k,
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1": scores["test_f1"].mean()
    })
```

Only k is changed while the other conditions remain fixed.

This makes the experiment controlled and allows us to observe how the neighborhood size affects performance.

	k	Accuracy	Precision	Recall	F1
0	3	0.8812	0.4568	0.2842	0.3502
1	5	0.8898	0.5215	0.2753	0.3603
2	7	0.8937	0.5581	0.2740	0.3671
3	9	0.8947	0.5716	0.2627	0.3597
4	11	0.8962	0.5885	0.2624	0.3625
5	15	0.8967	0.6003	0.2495	0.3522
6	19	0.8988	0.6283	0.2511	0.3584
7	25	0.8987	0.6332	0.2403	0.3480
8	31	0.8997	0.6514	0.2371	0.3471



Best k according to cross-validated F1: 7

- Experiment 3: Representation Investigation

The third experiment compares scaled and unscaled numerical features for KNN.

The experimental question is:

+ Does standardizing numerical features improve KNN performance compared with using unscaled numerical features?

```
preprocessor_scaled = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numeric_features
        ),
        (
            "cat",
            encoder_scaled,
            categorical_features
        )
    ]
)

knn_scaled = Pipeline([
    ("preprocess", preprocessor_scaled),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=best_k
        )
    )
])
```

```
preprocessor_unscaled = ColumnTransformer(
    transformers=[
        (
            "num",
            "passthrough",
            numeric_features
        ),
        (
            "cat",
            encoder_unscaled,
            categorical_features
        )
    ]
)

knn_unscaled = Pipeline([
    ("preprocess", preprocessor_unscaled),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=best_k
        )
    )
])
```

```
scaled_scores = cross_validate(
    knn_scaled,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)

unscaled_scores = cross_validate(
    knn_unscaled,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)
```

This experiment is directly connected to the central course principle that representation is not a neutral choice. Because KNN uses distances, changing feature scale can change which observations are considered neighbors.

	Representation	Accuracy	Precision	Recall	F1
0	Scaled numerical features	0.8937	0.5581	0.2740	0.3671
1	Unscaled numerical features	0.8902	0.5256	0.2664	0.3535

8. Results

- A majority-class baseline was established before evaluating the machine learning models.

	Metric	Baseline Score
0	Accuracy	0.8873
1	Precision	0.0000
2	Recall	0.0000
3	F1	0.0000

- The reason for the high Accuracy is that the dataset is strongly imbalanced. The majority class is no, so a strategy that always predicts no can obtain a high overall accuracy without actually identifying positive cases.

This result provides an important reference point. The question is not simply whether a machine learning model has high Accuracy, but whether it learns something useful beyond this simple strategy.

- The four models were then evaluated using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

The cross-validation results obtained in the project were:

	Model	Accuracy	Precision	Recall	F1
0	KNN	0.8937	0.5581	0.2740	0.3671
1	Decision Tree	0.8975	0.6072	0.2581	0.3620
2	Random Forest	0.8999	0.6643	0.2263	0.3372
3	Logistic Regression	0.8995	0.6564	0.2268	0.3370

These results show that Accuracy alone would lead to a different model choice from F1-score.

Random Forest has the highest Accuracy, while KNN has the highest F1-score.

This difference is important because the positive class is the minority class. Accuracy is strongly influenced by the large number of negative cases, while

Precision and Recall provide more information about the model's behavior on the positive class.

For this reason, F1-score is used as the primary metric for model selection, while Accuracy, Precision, Recall, and Confusion Matrix are all reported.

9. Model Comparison

The model comparison provides evidence that the choice of algorithm affects the result even when the models receive the same basic feature representation.

- Logistic Regression and Random Forest achieved relatively high Precision but lower Recall. This means that they were more selective when predicting the positive class, but they also missed a larger proportion of actual positive cases.

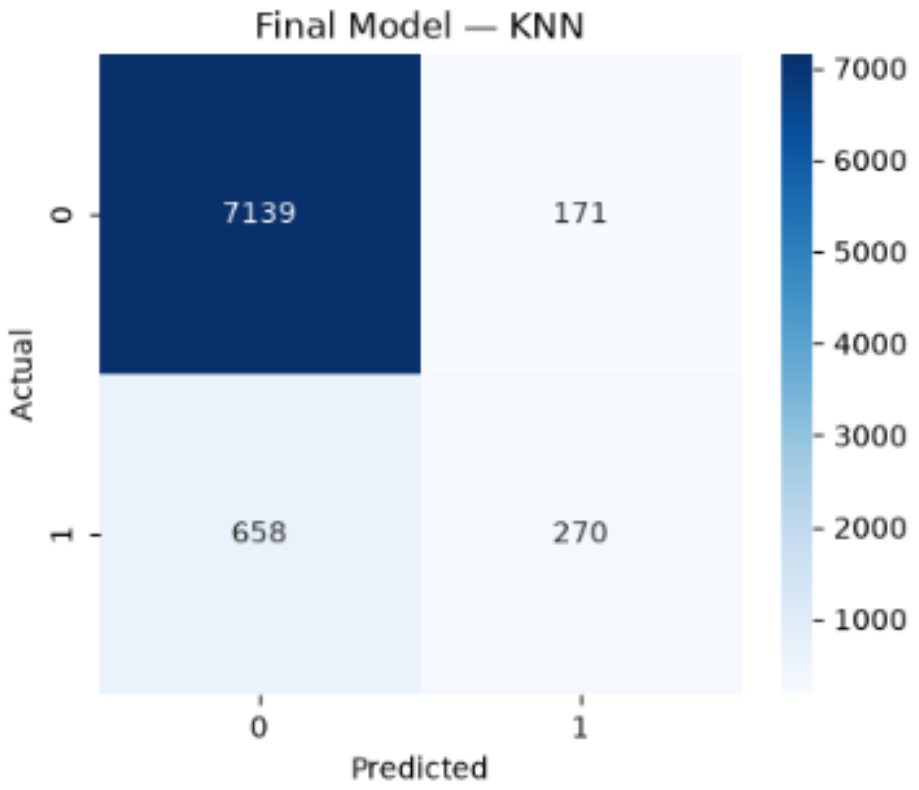
- KNN achieved the highest Recall among the four models and, as a result, achieved the highest F1-score.

- Decision Tree produced results between KNN and the two more conservative models.

The final model was therefore selected using cross-validation evidence rather than by simply choosing the model with the highest Accuracy.

In this project, the target is imbalanced and the application is interested in identifying potential subscribers. Therefore, both Precision and Recall are relevant. F1 is used as the main selection criterion because it summarizes the balance between these two metrics.

=> KNN achieved the highest cross-validated F1-score of 0.3671 and was therefore selected as the final model.



Importantly, the test set was not used to choose the final model. It was kept as an unseen dataset for the final evaluation.

```
Training set shape:
(32950, 19)

Test set shape:
(8238, 19)

Training target distribution:
y
0    0.887344
1    0.112656
Name: Proportion, dtype: float64

Test target distribution:
y
0    0.887351
1    0.112649
Name: Proportion, dtype: float64
```

10. Representation Analysis

- The project uses a feature-vector representation because the original data is structured and tabular.

This is exactly where traditional machine learning fits in the broader course progression:

Raw / Domain Data → Human-designed Features → Traditional ML Model → Prediction.

- The representation preserves numerical and categorical information that is relevant to the bank marketing problem. However, it does not preserve every aspect of the real-world situation.

For example, the representation does not naturally capture the full content of a conversation, the sequence of interactions over time, or complex relationships between entities.

This limitation is important because different information structures naturally suggest different representations. Records can be represented as feature vectors, images as tensors, relationships as graphs, and documents as embeddings.

- The representation experiment with KNN provides practical evidence for this idea. When numerical scaling is changed, the distances between observations can change, which can alter the neighbors selected by KNN and therefore change the prediction results.

11. Intelligent Application

- The final model was integrated into a small prediction application.

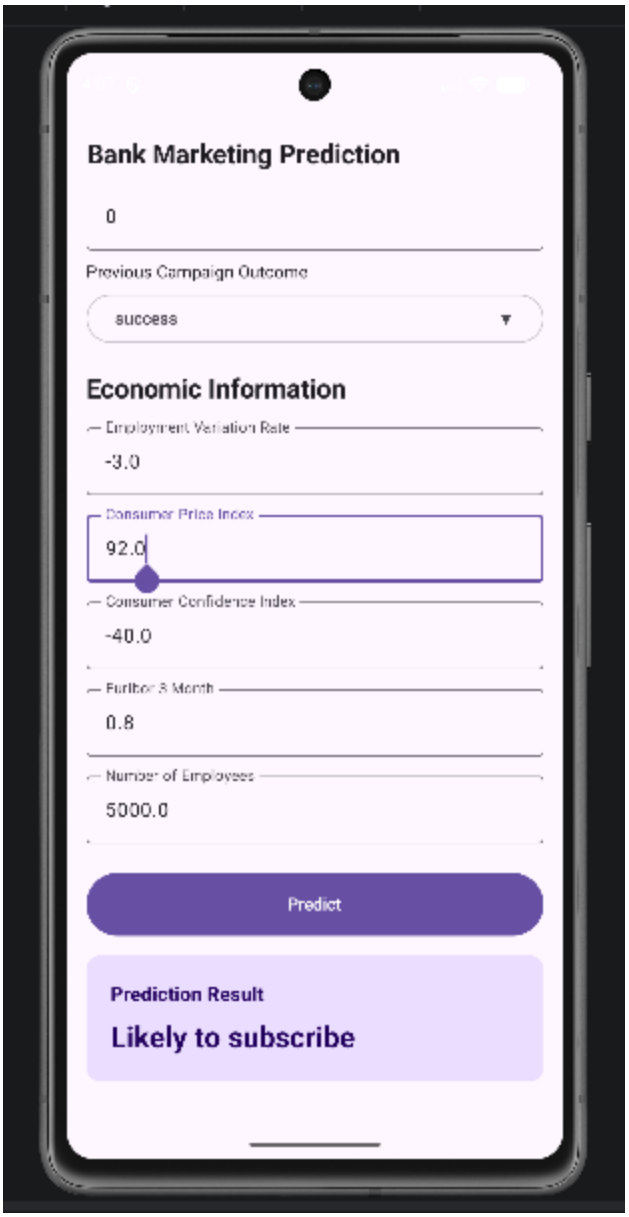
The application follows the complete path required by the assignment:

Input → Representation → Preprocessing → Model → Prediction → Output

- A new client record is entered into the application. The same feature columns and preprocessing pipeline used during model training are then applied to the new observation.

The final model produces the prediction, which is displayed to the user.

The application therefore represents the transition from a learned function to a usable system component.


Bank Marketing Prediction
0
Previous Campaign Outcome
success
Economic Information
Employment Variation Rate
-3.0
Consumer Price Index
92.0
Consumer Confidence Index
-40.0
Euribor 3 Month
0.8
Number of Employees
5000.0
Predict
Prediction Result
Likely to subscribe

MACHINE LEARNING APPLICATION

Bank Marketing Prediction

Predict whether a client is likely to subscribe to a term deposit.

01 Customer Information

Basic information about the client

Age: 30

Job: student

Marital Status: single

Education: high.school

Credit Default: no

Housing Loan: no

Personal Loan: no

02 Campaign Information

Information about the marketing campaign

Contact Type: cellular

Month: mar

Day of Week: thu

Campaign Contacts: 1

Days Since Previous Contact: 999

Previous Contacts: 0

Previous Campaign Outcome: success

03 Economic Information

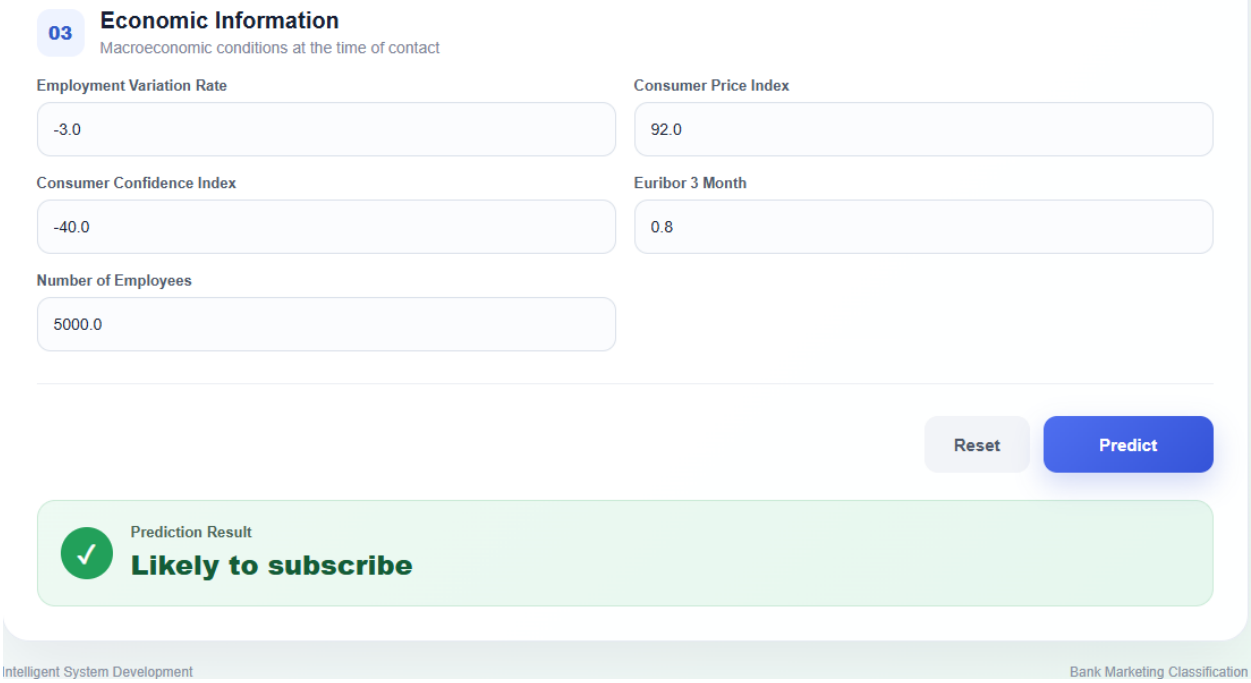
Macroeconomic conditions at the time of contact

Employment Variation Rate: -3.0

Consumer Price Index: 92.0

Consumer Confidence Index: -40.0

Euribor 3 Month: 0.8



12. Limitations

- The system has some limitations.

+ First, the model depends strongly on the quality and representativeness of the training dataset. If future customers behave differently from the historical observations, performance may decrease.

+ Second, the target is imbalanced. This makes Accuracy less informative and requires careful interpretation of Precision, Recall, and F1.

+ Third, the feature-vector representation is limited to the information explicitly selected for the model. Information that is not represented cannot be directly used by the learner.

+ Finally, the system is based on traditional machine learning. It learns a mapping from the available feature representation to the target, but it does not independently reason about the wider environment.

Therefore, this project should be understood as the first stage of intelligent system development rather than a complete intelligent system in the broadest sense.

13. Reflection

- The most important lesson from this project is that developing a machine learning system is not the same as calling fit on a model.

+ The first important issue is representation. The model only receives the computational representation created from the raw data. Therefore, the representation determines what information is available to the learner.

+ The second lesson is that different algorithms can learn different relationships from the same representation. Logistic Regression learns a weighted relationship, KNN uses neighborhood similarity, Decision Tree uses recursive decision rules, and Random Forest combines many decision trees.

+ The third lesson is that experiments should answer questions. Changing k in KNN or changing numerical scaling is useful only when the experiment helps us understand how those changes affect the model.

+ The fourth lesson is that evaluation must be connected to the problem. The baseline showed that high Accuracy alone can be misleading for this dataset. A model must be evaluated using metrics that reflect the behavior we actually care about.

14. Conclusion

This project developed a small intelligent system for predicting term-deposit subscription using structured Bank Marketing data and traditional machine learning.

A feature-vector representation was designed from the structured dataset. Numerical features were standardized and categorical features were one-hot encoded. Four traditional machine learning models were then trained and compared.

A baseline established that a simple majority-class strategy could obtain high Accuracy while failing to identify the positive class. This demonstrated why evaluation must go beyond Accuracy.

Three controlled experiments were conducted. The first compared different learning mechanisms. The second investigated the KNN hyperparameter k. The third investigated the effect of feature representation through numerical scaling.

The final model was then connected to a small application, completing the transition from data and representation to a usable prediction system.

The project represents the first stage of the course journey:

Structured Data → Feature Vectors → Traditional ML → Application.

The central lesson is that intelligent-system development is not only about training a model. It is about understanding the problem, deciding what to represent, selecting an appropriate learning mechanism, designing experiments, evaluating evidence, and turning the learned model into something that can be used.